IMS ENGINEERING COLLEGE

LABORATORY MANUAL
WEB TECHNOLOGY
(BCS-552)

B.TECH – III YEAR
(ODD SEM 2024-2025)

Name	
Roll No.	
Section-Batch	

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

IMS ENGINEERING COLLEGE

(Affiliated to Dr A P J Abdul Kalam Technical University, Lucknow)
Approved by AICTE - Accredited by NAAC – 'A' Grade
NH#24Adhyatmik Nagar, Ghaziabad, UP, India
www.imsec.ac.in

Vision and Mission of the Institute and Department

Vision of the Institute

To make IMSEC an Institution of Excellence for empowering students through technical education coupled with incorporating values and developing engineering acumen for innovations and leadership skills for the betterment of society.

Mission of the Institute

Mission 1: To promote academic excellence by continuous learning in core and emerging Engineering areas using innovative teaching and learning methodologies.

Mission 2: To inculcate values and ethics among the learners.

Mission 3: To promote industry interactions and produce young entrepreneurs.

Mission 4: To create a conducive learning and research environment for life-long learning to develop the students as technology leaders and entrepreneurs for addressing societal needs.

Vision of the Department

To provide globally competent professionals in the field of Computer Science & Engineering embedded with sound technical knowledge, aptitude for research and innovation, and nurture future leaders with ethical values to cater to the industrial & societal needs.

Mission of the Department

Mission 1: To provide quality education in both the theoretical and applied foundations of Computer Science & Engineering.

Mission 2: Conduct research in Computer Science & Engineering resulting in innovations thereby nurturing entrepreneurial thinking.

Mission 3: To inculcate team building skills and promote life-long learning with a high societal and ethical values.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

Graduate Will:

PEO1: Possess core theoretical and practical knowledge in Computer Science and Engineering for successful career development in industry, pursuing higher studies or entrepreneurship.

PEO2: Ability to imbibe life long learning for global challenges to impact society and environment.

PEO3: To demonstrate work productivity with leadership and managerial skills having ethics and human value in progressive career path.

PEO4: To exhibit communication skill and collaborative skill plan and participate in multidisciplinary fields of Computer Science & Engineering.

PROGRAMME SPECIFIC OUTCOME (PSOs)

PSO1: To analyze and demonstrate, the recent engineering practices, ethical values and strategies in real time world problems to meet the challenges for the future.

PSO2: To develop adaptive computing system using computational intelligence strategies and algorithmic design to address diverse challenges in data analysis and machine learning.

PROGRAM OUTCOMES

Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, andenvironmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

GENERAL LABORATORY INSTRUCTIONS

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, comeprepared to the lab with the synopsis / program / experiment details.
3. Student should enter into the laboratory with:
    - Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session.
    - Laboratory Record updated up to the last session experiments and other utensils (if any) needed in the lab.
    - Proper Dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results / output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high end branded systems, which should be utilized properly.
8. Students / Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviors with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out; if anybody found loitering outside the lab / class without permission during working hours will be treated seriously and punishedappropriately.
10. Students should LOG OFF/ SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in allaspects. He/she must ensure the system / seat is kept properly.

DETAILS OF THE EXPERIMENTS CONDUCTED

(TO BE USED BY THE STUDENTS IN THEIR RECORDS)

INDEX

S.No	TITLE OF THE EXPERIMENT	DATE OF SUBMISSION	FACULTY SIGNATURE
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

STUDY AND EVALUATION SCHEME

Course Code	Course Name	Teaching Scheme			Credits Assigned			
		Theory	Practical	Tutorial	Theory	Practical	Tutorial	Total
BCS-502(T) BCS-552 (P)	**Web Technology Lab**	**03 (70 Marks)**	**02 (50 Marks)**	**01**	**03**	**01**	**01**	**05**

IMS Engineering College

NH-09, Adhyatmik Nagar, Near Dasna, Distt. Ghaziabad, U.P.

Tel: (0120) 4940000

Department of Computer Science and Engineering

Course Name: Web Technology Lab **Course Code: BCS-552**

Semester / Year: 5Th /3rd **NBA Code: C314**

COURSE OUTCOMES		Bloom's Level
C314.1	Understand the fundamental concepts of web development, including the history, protocols, and tools. Apply HTML and XML in the development of web projects.	K3, K6
C314.2	Apply CSS for designing and styling web pages, including the use of CSS properties, styling elements, and advanced techniques for creating responsive web sites.	K2, K3
C314.3	Develop interactive web applications using JavaScript and AJAX, with a focus on scripting documents, forms, and networking concepts such as internet addressing and TCP/IP sockets.	K3, K6
C314.4	Design and implement server-side applications using Enterprise Java Beans (EJB) and Node.js, including the creation of JavaBeans, RESTful APIs, and database operations with MongoDB.	K2, K4, K6
C314.5	Implement web server functionality using Servlets and Java Server Pages (JSP), focusing on handling HTTP requests, session tracking, and utilizing custom tag libraries for dynamic web content.	K2, K3, K4

CO-PO Matrix

Course Outcome	PO 1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
C314.1	3	3	3	3	3	-	-	-	3	1	2	3	3	3
C314.2	3	3	3	3	3	-	-	-	3	1	2	3	3	3
C314.3	3	3	3	3	3	-	-	-	3	1	2	3	3	3
C314.4	3	3	3	3	3	-	-	-	3	1	2	3	3	3
C314.5	3	3	3	3	3	-	-	-	3	1	2	3	3	3
Avg	3.00	3.00	3.00	3.00	3.00	-	-	-	3.00	1.00	2.00	3.00	3.00	3.00

LIST OF PROGRAMS

Subject: Web Technology Lab Code: BCS-552

S.NO.	NAME OF EXPERIMENT	MAPPING WITH CO
1.	Write HTML program for designing your institute website. Display departmental information of your institute on the website.	C314.1
2.	Write HTML program to design an entry form for student details/employee information/faculty details.	C314.1
3.	Develop a responsive website using CSS and HTML. Website may be for tutorial/blogs/commercial website.	C314.2
4.	Write programs using HTML and Java Script for validation of input data.	C314.3
5.	Write a program in XML for creation of DTD, which specifies set of rules. Create a style sheet in CSS/ XSL & display the document in internet explorer.	C314.2
6.	Create a Java Bean for Employee information (EmpID, Name, Salary, Designation and Department).	C314.4
7.	Build a command-line utility using Node.js that performs a specific task, such as converting text to uppercase, calculating the factorial of a number, or generating random passwords.	C314.4
8.	Develop a script that uses MongoDB's aggregation framework to perform operations like grouping, filtering, and sorting. For instance, aggregate user data to find the average age of users in different cities.	C314.4
9.	Assume four users user1, user2, user3 and user4 having the passwords pwd1, pwd2, pwd3 and pwd4 respectively. Write a servlet for doing the following: 1. Create a Cookie and add these four user id's and passwords to this Cookie. 2. Read the user id and passwords entered in the Login form and authenticate with the values available in the cookies.	C314.4
10.	Create a table which should contain at least the following fields: name, password, email-id, phone number Write Servlet/JSP to connect to that database and extract data from the tables and display them. Insert the details of the users who register with the web site, whenever a new user clicks the submit button in the registration page.	C314.4
11.	Write a JSP which insert the details of the 3 or 4 users who register with the web site by using registration form. Authenticate the user when he submits the login form using the user name and password from the database.	C314.5
12.	Design and implement a simple shopping cart example with session tracking API.	C314.5

EXPERIMENT NO. - 1

AIM: Write HTML program for designing your institute website. Display departmental information of your institute on the website.

Procedure:

HTML stands for Hyper Text Markup Language. HTML is the computer language that is used to create documents for display on the Web. Many editors exist to create Web Pages – Word, FrontPage, and Dream Weaver are just a few. Nevertheless, each of these software programs (editors) performs the exact same task – they all generate HTML language.The HTML language consists of a series of HTML tags. Learning HTML involves finding out what tags are used to mark the parts of a document and how these tags are used in creating an HTML document. Tags are instructions that tell your browser what to show on a Web page. They break up your document into basic sections. All tags start with a < (left bracket) and end with a > (right bracket).

Basic HTML Tags

<HTML></HTML>

This tag tells your browser that the file contains HTML-coded information. All html tags must be placed between the open <HTML> tag and the closed tag </HTML>The file extension .html also indicates the document is an HTML document. All html documents MUST be saved with the .html file extension.

<HEAD></HEAD>

The head tag identifies the first part of your HTML-coded document. The title tag (explained below) must be places between the open <HEAD> tag and the closed </HEAD> tag.

<BODY></BODY>

The largest part of your HTML document is the body, which contains the content of your document (displayed within the text area of your browser window). All HTML tags that pertain to the body of your HTML document must be places between the open <BODY> tag and the closed </BODY> tag. The tag has attributes which you can use to set the colors of your background, text, links, and also to include your own background image. They are as follows:

BGCOLOR="white" Sets the background color (other color names: red, black, blueetc)

- TEXT="black" Sets the body text color
- LINK="blue" Sets the unvisited hypertext links
- VLINK ="purple" Sets the visited hypertext links
- ALINK="red" Sets the active hypertext links (the color of the hypertext link when you have your mouse button depressed)
- BACKGROUND Let you use an image as the background <background=Body attributes are used as part of the open <body> tag. For example:<BODY BGCOLOR = "white" TEXT = "black" LINK = "blue" VLINK = "purple" ALINK ="red">

Code/Method:

To create a complete lab manual that connects all these pages together, you can link the pages using HTML navigation, ensuring seamless transitions between different sections (labs, sections, semesters, and faculty information). Additionally, you can include comments and explanations in the code to guide students or readers about the structure.

Here's how to connect all the files together into a **Lab Manual** format with comments and explanations.

1. **Main Page (index.html)**

This is the entry point for your lab manual. It will provide a simple navigation menu for users to access different sections.

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Institute Lab Manual</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<!-- Header of the website -->
  <header>
    <h1>Welcome to the CSE1 Lab Manual</h1>
    <p>Navigate through the labs, sections, semester info, and faculty details.</p>
  </header>

  <!-- Navigation menu linking to different sections of the lab manual -->
  <nav>
    <a href="labs.html">Labs</a>
    <a href="sections.html">Sections</a>
    <a href="semesters.html">Semesters</a>
    <a href="faculty.html">Faculty</a>
  </nav>

  <!-- Main content explaining what this lab manual covers -->
  <main>
    <h2>Department Information</h2>
    <p>This is the landing page for the Computer Science and Engineering department's lab manual. Here, you can
      navigate to learn about labs, sections, semester-wise subjects, and the faculty involved in teaching these
      courses.</p>
    <p><b>Vision & Mission:</b>Describe the department's vision to develop skilled professionals and its mission to
      provide a high-quality
```

```
        education and foster research in the field.</p>
      <p><b>Goals:</b> List out the primary goals, like enhancing employability, promoting innovation, and
        contributing to industry needs.</p>
  </main>

  <!-- Footer with additional information -->
  <footer>
    <p>&copy; 2024 CSE Department, Your Institute Name</p>
  </footer></body>
</html>
```

2. **Labs Page (labs.html)**

This page gives information about the various labs in the department. The navigation links at the top allow the user to switch between different sections of the lab manual.

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Labs - CSE Lab Manual</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <!-- Header for the labs page -->
  <header>
    <h1>Computer Science Labs</h1>
  </header>

  <!-- Navigation bar -->
  <nav>
    <a href="index.html">Home</a>
    <a href="sections.html">Sections</a>
    <a href="semesters.html">Semesters</a>
    <a href="faculty.html">Faculty</a>
  </nav>

  <!-- Main content about the labs -->
  <main>
    <h2>Labs Information</h2>
    <p>Our department offers the following labs for hands-on learning:</p>
    <ul>
      <li>Programming Lab</li>
      <li>Network Lab</li>
      <li>Database Lab</li>
      <li>AI and Machine Learning Lab</li>
      <li>Software Engineering Lab</li>
```

```
        </ul>
    </main>

    <!-- Footer -->
    <footer>
        <p>&copy; 2024 CSE Department, Your Institute Name</p>
    </footer>

</body>
</html>
```

3. **Sections Page (sections.html)**

This page explains the various sections in the department, typically for different groups of students.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sections - CSE Lab Manual</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Header for the sections page -->
    <header>
        <h1>Sections Information</h1>
    </header>

    <!-- Navigation bar -->
    <nav>
        <a href="index.html">Home</a>
        <a href="labs.html">Labs</a>
        <a href="semesters.html">Semesters</a>
        <a href="faculty.html">Faculty</a>
    </nav>

    <!-- Main content about the sections -->
    <main>
        <h2>Available Sections</h2>
        <p>The department has multiple sections catering to different types of students:</p>
        <ul>
            <li>Section A - Advanced Group</li>
            <li>Section B - Regular Group</li>
            <li>Section C - Students Needing Extra Guidance</li>
        </ul>
    </main>

    <!-- Footer -->
```

```
    <footer>
        <p>&copy; 2024 CSE Department, Your Institute Name</p>
    </footer>

</body>
</html>
```

4. **Semesters Page (semesters.html)**

This page provides semester-wise information on the subjects offered.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Semesters - CSE Lab Manual</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Header for the semesters page -->
    <header>
        <h1>Semester Information</h1>
    </header>

    <!-- Navigation bar -->
    <nav>
        <a href="index.html">Home</a>
        <a href="labs.html">Labs</a>
        <a href="sections.html">Sections</a>
        <a href="faculty.html">Faculty</a>
    </nav>

    <!-- Main content about the semester-wise subjects -->
    <main>
        <h2>Subjects by Semester</h2>
        <p>The Computer Science department offers the following subjects across semesters:</p>
        <ul>
            <li><strong>Semester 1:</strong> Programming Fundamentals, Mathematics, Physics</li>
            <li><strong>Semester 2:</strong> Data Structures, Discrete Mathematics, Computer
Organization</li>
            <li><strong>Semester 3:</strong> Algorithms, Operating Systems, Networks</li>
            <!-- More semesters can be added as required -->
        </ul>
    </main>

    <!-- Footer -->
    <footer>
        <p>&copy; 2024 CSE Department, Your Institute Name</p>
```

```
    </footer>

</body>
</html>
```

5. **Faculty Page (faculty.html)**

This page provides the details of the faculty members in the department.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Faculty - CSE Lab Manual</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Header for the faculty page -->
    <header>
        <h1>Faculty Information</h1>
    </header>

    <!-- Navigation bar -->
    <nav>
        <a href="index.html">Home</a>
        <a href="labs.html">Labs</a>
        <a href="sections.html">Sections</a>
        <a href="semesters.html">Semesters</a>
    </nav>

    <!-- Main content about the faculty -->
    <main>
        <h2>Our College(IMSEC) Faculty</h2>
        <ul>
            <li>Dr. Sonali Mathur - Professor and Head of Department</li>
            <li>Dr. Amit Chugh  - Associate Professor</li>
            <li>Dr. Tejaswi Khanna – Assistant Professor</li>
            <!-- Add more faculty as necessary -->
        </ul>
    </main>

    <!-- Footer -->
    <footer>
        <p>&copy; 2024 CSE Department, Your Institute Name</p>
    </footer>

</body>
</html>
```

```
### 6. **CSS File (style.css)**

This is the shared CSS file that gives a consistent look and feel to all pages.

```css
body {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
    margin: 0;
    padding: 0;
}

header {
    background-color: #004080;
    color: white;
    text-align: center;
    padding: 1rem;
}

nav {
    background-color: #333;
    overflow: hidden;
}

nav a {
    float: left;
    color: white;
    text-align: center;
    padding: 14px 20px;
    text-decoration: none;
}

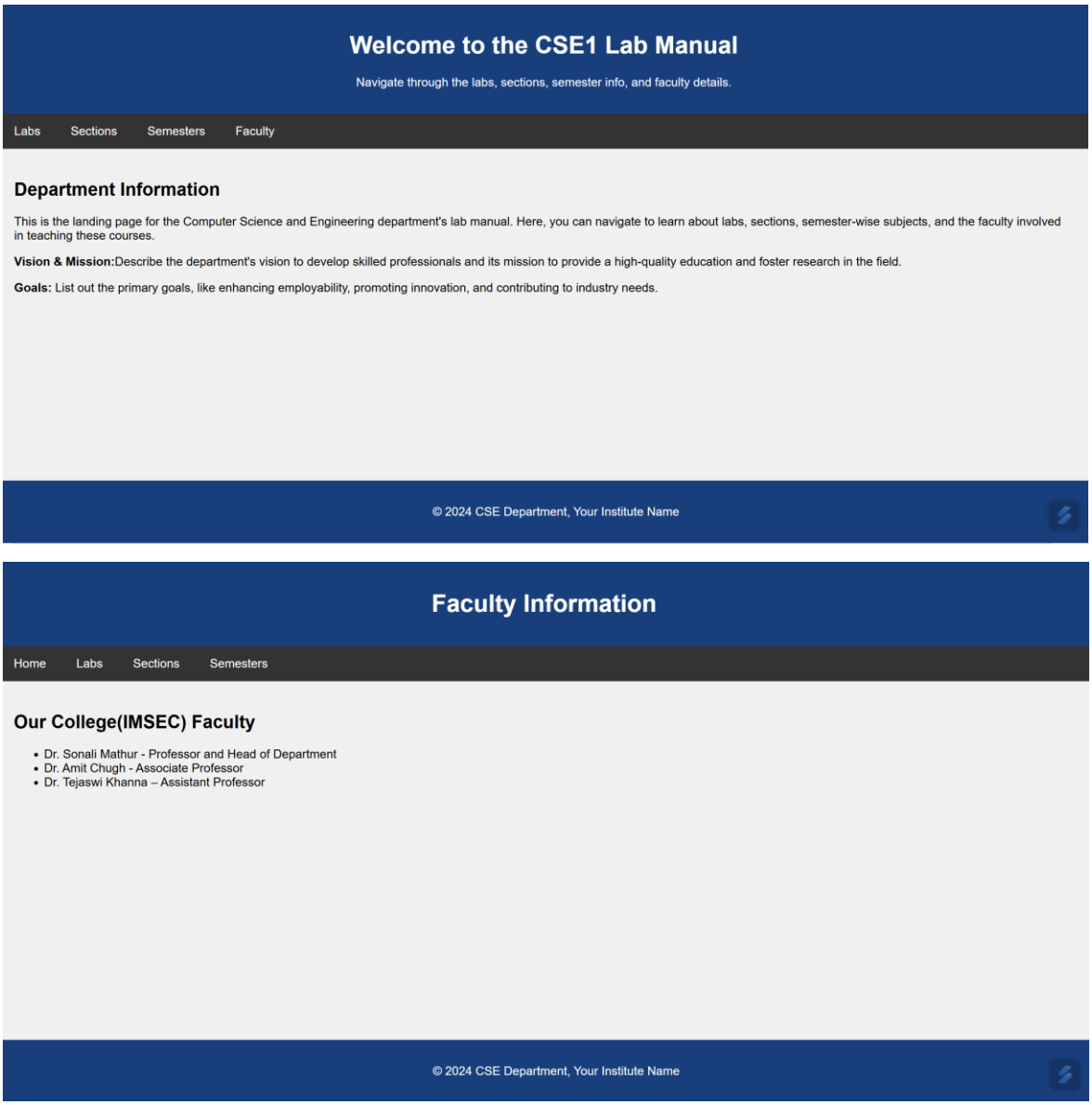
nav a:hover {
    background-color: #ddd;
    color: black;
}

main {
    padding: 20px;
    margin: 0 auto;
}

footer {
    background-color: #004080;
    color: white;
    text-align: center;
    padding: 1rem;
    position: absolute;
    width: 100%;
    bottom: 0;
}
```
```

```

## Output:







Viva Questions:

1. What is HTML Tag ?
2. What is purpose of <li>tag ?
3. Why we use tag <p>?
4. What is Image tag<img>?
5. What is Anchor tag<a>?
```

EXPERIMENT NO. - 2

AIM- Write HTML program to design an entry form for student details/employee information/faculty details.

Procedure:

HTML Forms -HTML Forms are required to collect different kinds of user inputs, such as contact details like name, email address, phone numbers, or details like credit card information, etc. Forms contain special elements called controls like inputbox, checkboxes, radio-buttons, submit buttons, etc. Users generally complete a form by modifying its controls e.g. entering text, selecting items, etc. and submitting this form to a web server for processing. The <form> tag is used to create an HTML form. Here's a simple example of a login form:

Input Element- This is the most commonly used element within HTML forms.It allows you to specify various types of user input fields, depending on the type attribute. An input element can be of type text field, checkbox, password field, radio button, submit button, reset button, etc. and several new input types introduced in HTML5. The most used input types are described below.

Text Fields- Text fields are one line areas that allow the user to input text.Single-line text input controls are created using an <input> element, whose type attribute has a value of text. Here's an example of a single-line text input used to take username:

Password Field- Password fields are similar to text fields. The only difference is; characters in a password field are masked i.e. shown as asterisks or dots. This is to prevent others from reading the password on the screen. This is also a single-line text input controls created using an <input> element whose type attribute has a value of password. Here's an example of a single-line password input used to take user password:

Radio Buttons- Radio buttons are used to let the user select exactly one option from a pre-defined set of options. It is created using an <input> element whose type attribute has a value of radio.

Checkboxes- Checkboxes allows the user to select one or more option from a pre-defined set of options. It is created using an <input> element whose type attribute has a value of checkbox.

File Select box- The file fields allow a user to browse for a local file and send it as an attachment to the form data. It normally rendered as a text box with a button that enables the user to browse for a file. However, the user can also type the path and name of the file in the text box. This is also created using an <input> element, whose type attribute value is set to file.

Textarea- Textarea is a multiple-line text input control that allows a user to enter more than one line of text. Multi-line text input controls are created using an <textarea> element.

Select Boxes- A select box is a drop down list of options that allows user to select one or more option from a pull-down list of options. Select box is created using the <select>

element and <option> element. The option elements within the <select> element define each list item.

Submit and Reset Buttons- A submit button is used to send the form data to a web server. When submit button is clicked the form data is sent to the file specified in the form's action attribute to process the submitted data. A reset button resets all the forms control to default values.

Code/Method:

###**HTML File (index.html)**

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
   <link rel="stylesheet" href="form.css">
</head>

<body>
   <form>
      <div class="main">
         <h1>Entry form</h1><br>
         <label class="name">Select Entry Type:</label>
         <select id="entryType" name="entryType" required>
            <option value="">--Select--</option>
            <option value="student">Student</option>
            <option value="employee">Employee</option>
            <option value="faculty">Faculty</option>
         </select><br>

         <div class="name">
            <div>
               <label>Name :</label>
            </div>
            <input type="text" placeholder="first name">
            <input type="text" placeholder="middle name">
            <input type="text" placeholder="last name">
         </div>
         <div class="email">
            <div>
               <label>Email :</label>
            </div>
            <input type="text" placeholder="enter your email">
         </div>
         <div class="email">
            <div>
               <label>DOB :</label>
            </div>
            <input type="date" placeholder="enter your email">
         </div>
```

```
            <div class="email">
                <div>
                    <label>Phone Number:</label>
                </div>
                <input type="tel" placeholder="XXXXXXXXXX">
            </div>
            <div class="gender">
                <div>
                    <label>Gender :</label>
                </div>
                <input type="radio" name="gender">Male
                <input type="radio" name="gender">Female
            </div>
            <div class="email">
                <div>
                    <label>Address :</label>
                </div>
                <textarea name="message" id="message" placeholder="enter your message here" rows="3"
                    cols="50"></textarea>
            </div>
            <div class="email">
                <div>
                    <label>Student Details:</label>
                </div>
                <textarea name="message" id="message" placeholder="enter your message here" rows="3"
                    cols="50"></textarea>
            </div>

            <button>SUBMIT</button>
        </div>
    </form>
</body>

</html>
```

###**CSS File (style.css)**

```
* {
    margin: 0;
    padding: 0;
}

body {
    background-color: white;
}

h1 {
    color: black;
    text-align: center;
}

.main {
    width: 50%;
```

```
    border: 2px solid rgb(52, 193, 244);
    border-radius: 10px;
    padding: 20px 50px;
    margin: 20px 300px;
    background-color: rgb(113, 206, 227);
}

.name,
.email,
.phone,
.gender {
    color: black;
    font-size: larger;
    padding: 10px 0px 10px 0px;
    font-weight: 600;
}

.name input {
    height: 30px;
    border-radius: 10px;
    width: 30.9%;
    padding: 0px 0px 0px 10px;
    margin-top: 10px;
    font-size: medium;
}

#entryType {
    height: 30px;
    border-radius: 10px;
    width: 98%;
    padding: 0px 0px 0px 10px;
    margin-top: 10px;
    margin-left: 10px;
    font-size: medium;
}

.email input {
    height: 30px;
    border-radius: 10px;
    width: 97%;
    padding: 0px 10px 0px 10px;
    margin-top: 10px;
    font-size: medium;
}

.gender input {
    height: 20px;
    border-radius: 10px;
    width: 10%;
    align-items: center;
    margin-top: 10px;
}

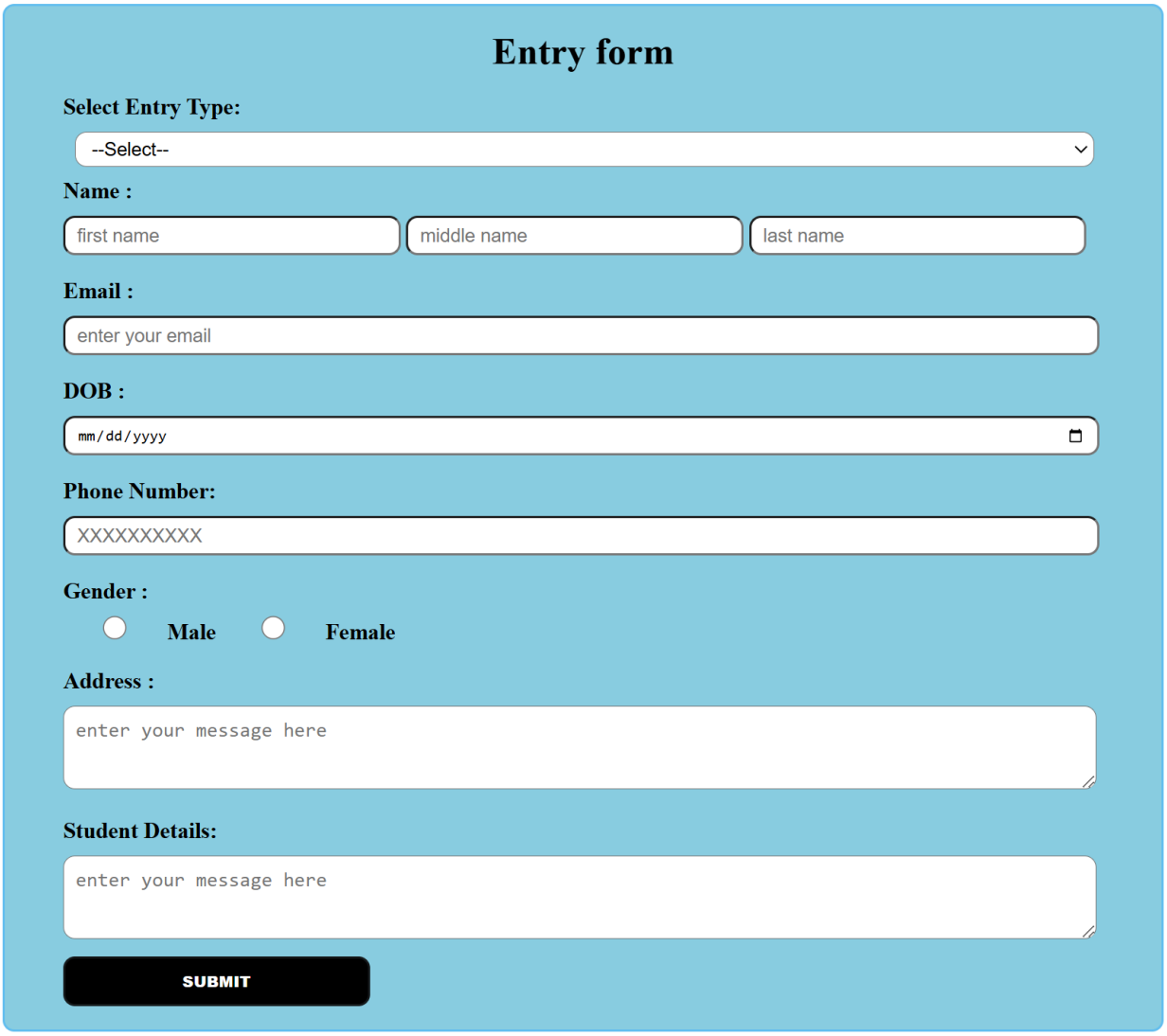
.email textarea {
```

```
    height: 30px;
    border-radius: 10px;
    width: 97%;
    height: 10%;
    padding: 10px 10px 0px 10px;
    margin-top: 10px;
    font-size: 17px;
}

button {
    padding: 10px 100px 10px 100px;
    border-radius: 10px;
    border: 2px solid rgb(20, 25, 27);
    color: white;
    background-color: black;
    transition: 0.9s all ease;
    font-weight: 800;
}

button:hover {
    background-color: rgb(33, 176, 228);
    color: black;
}
```

Output:



Viva Questions-

1. What is element and attributes in HTML?
2. What is purpose of <td> and <tr> Tag?
3. What is Heading Tag<h1>..... <h6>?
4. What is Table Tag?
5. Define row * column <tr><td>.

EXPERIMENT NO. - 3

AIM- Develop a responsive website using CSS and HTML. Website may be for tutorial/blogs/commercial website.

Objective:

To design and implement a responsive website that adjusts its layout based on the screen size of the device, such as a desktop, tablet, or mobile phone.

Responsive web design refers to building websites that adapt to different screen sizes and device orientations to ensure a consistent user experience. The key to responsiveness lies in the use of flexible grid layouts, media queries, and dynamic content resizing.

Procedure:

Step 1: Set up the basic HTML structure for your website.

Step 2: Implement CSS for responsiveness using media queries.

Step 3: Test the website layout on various devices (desktop, tablet, and mobile).

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple Responsive Page</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <!-- Header Section -->
    <header>
        <h1>Welcome to Our Grocery Website</h1>
        <p>Providing quality products for all your needs.</p>
    </header>

    <!-- Card Section -->
    <section id="cards">
        <div class="card">
            <img src="https://media.istockphoto.com/id/183422512/photo/fresh-red-apples-on-white-
background.jpg?s=612x612&w=0&k=20&c=OmfmcQLh3mwp4pFVQn9Sr2U8VCyIgGtV6fuaDmd3Y
rk="
```

```
            width="300px" height="300px">
          <h2>Apple</h2>
          <p>We offer top-notch solutions tailored to your needs.</p>
        </div>
        <div class="card">
          <img src="https://media.istockphoto.com/id/1222018207/photo/pouring-milk-into-a-drinking-
glass.jpg?s=612x612&w=0&k=20&c=eD4YHoSjKIYSPDgnM2OgWD_HVH2IcmjZSRq7IjUnH6M="
            width="300px" height="300px">
          <h2>Milk</h2>
          <p>Expert services to help your business grow.</p>
        </div>
        <div class="card">
          <img src="https://www.shutterstock.com/image-photo/nescafe-coffee-pack-600nw-
2464133955.jpg" width="300px"
            height="300px">
          <h2>Nescafe</h2>
          <p>Innovative approaches for the modern market.</p>
        </div>
    </section>

    <!-- Content Section -->
    <section id="content">
        <h2>About Us</h2>
        <p>Our company is committed to delivering high-quality solutions and exceptional service to meet
all your
            requirements. We pride ourselves on our expertise, reliability, and customer satisfaction.</p>
    </section>

    <!-- Footer Section -->
    <footer>
        <p>&copy; 2024 Your Company Name. All rights reserved.</p>
    </footer>

</body>

</html>
```

Styles.css

```
* {

    margin: 0;

    padding: 0;

    box-sizing: border-box;

}
```

```
body {
    font-family: Arial, sans-serif;
    color: #333;
    line-height: 1.6;
}
/* Header */
header {
    text-align: center;
    padding: 2em;
    background-color: #f5f5f5;
}
header h1 {
    font-size: 2em;

header p {
    margin-top: 0.5em;
    color: #666;
}
/* Cards Section */
#cards {
    display: flex;
    flex-direction: column;
    gap: 1em;
    padding: 2em;
}
.card {
```

```
    padding: 1.5em;
    border-radius: 8px;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    background-color: #fff;
    text-align: center;
}
/* Content Section */
#content {
    text-align: center;
    padding: 2em;
}
#content h2 {
    font-size: 1.8em;
    margin-bottom: 0.5em;
}
/* Footer */
footer {
    text-align: center;
    padding: 1em;
    background-color: #333;
    color: #fff;
    font-size: 0.9em; }
/* Responsive Styles */
@media (min-width: 768px) {
    #cards {
        flex-direction: row;
```

```
        justify-content: space-between;
    }
    .card {
        flex: 1;
        margin: 0 1em;
    }
}
```

- **Meta Tags for Responsiveness**:
  The `<meta name="viewport" content="width=device-width, initial-scale=1.0">` tag ensures the webpage scales properly on different devices by setting the viewport width to the device's width.

- **Basic HTML Structure**:
  The website is divided into several sections: `header`, `main`, and `footer`. Inside the `main` tag, we have several `section` tags that represent different parts of the website: Home, About, Blog, and Contact.

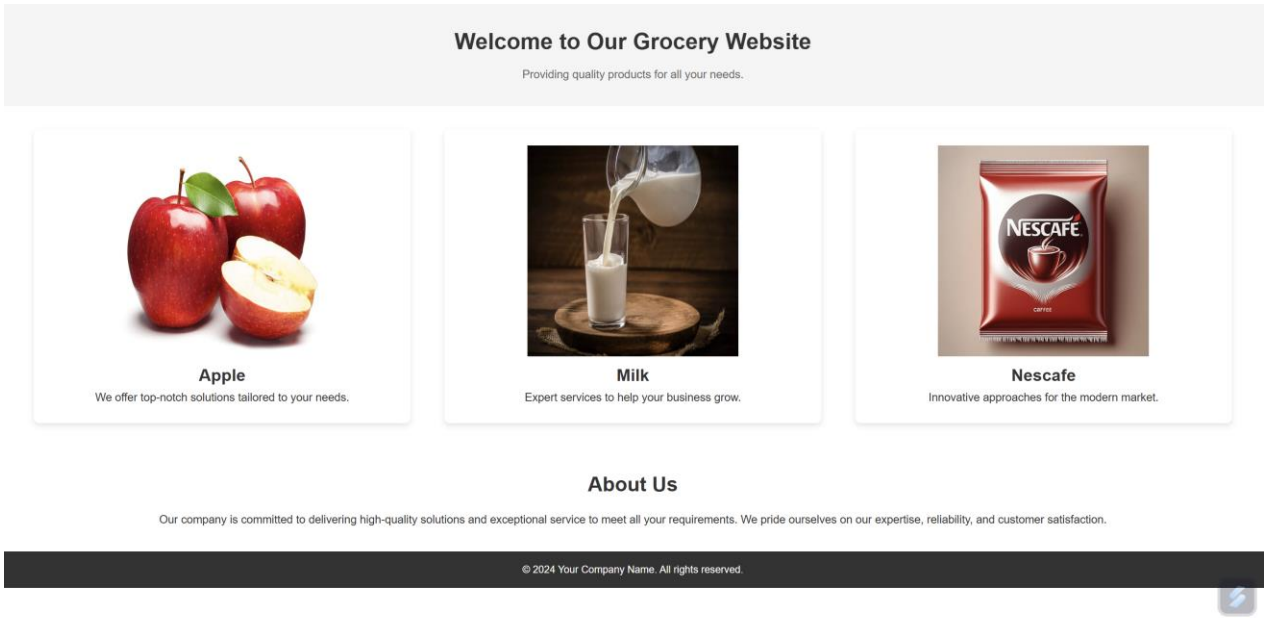
- **Navigation Bar**:
  The `nav` element contains an unordered list of links that allow users to navigate between different sections of the webpage. On smaller screens, the navigation adjusts to become vertically aligned.

- **Form in the Contact Section**:
  A simple form is created using the `form` element to collect the user's name and email address, with validation for required fields using the `required` attribute.

- **Media Queries for Responsive Design**:

- The first media query (`max-width: 768px`) adjusts the layout for tablet-sized devices by making the navigation items stack vertically and reducing padding.
- The second media query (`max-width: 480px`) is used for mobile screens. It further reduces the font sizes and margin/padding to fit smaller screens.

Output:



Viva Questions:

1. What is the difference between `<div>` and `<section>` tags in HTML?
2. What is the role of the `<meta charset="UTF-8">` tag in an HTML document?
3. Explain the box model in CSS and how margins, padding, and borders affect the layout of elements.
4. What are the advantages of using external CSS compared to inline or internal CSS?
5. What is a media query in CSS?

EXPERIMENT NO. - 4

AIM- Write programs using HTML and Java Script for validation of input data.

Procedure:

To validate user input in an HTML form using JavaScript. The form will check for required fields, email format, and specific password rules before submitting the data.

Form validation is an important part of web development, ensuring that users provide correct and complete information before submitting data to the server. Client-side validation using JavaScript allows for immediate feedback without requiring server communication, improving the user experience. Common validation techniques include checking for empty fields, validating email format, and password strength.

Step 1: Create an HTML form with fields for a username, email, and password.

Step 2: Write JavaScript functions to validate the form data (checking if fields are empty, ensuring the email is valid, and the password meets specific criteria).

Step 3: Link the JavaScript file or embed the JavaScript code in the HTML file to ensure the validation occurs when the form is submitted.

Step 4: Test the form by submitting invalid and valid data to observe the validation process.

Code/Method: form_validation.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Form Validation</title>
   <style>
      /* Basic styling for form and error messages */
      body {
         font-family: Arial, sans-serif;
         margin: 50px;
      }
      .error {
         color: red;
         font-size: 0.9em;
      }
      form {
         background-color: #f9f9f9;
         padding: 20px;
         border-radius: 5px;
         max-width: 400px;
         margin: auto;
      }
      input[type="text"], input[type="email"], input[type="password"] {
         width: 100%;
         padding: 10px;
```

```
            margin: 10px 0;
        }
        input[type="submit"] {
            padding: 10px 20px;
            background-color: #4CAF50;
            color: white;
            border: none;
            cursor: pointer;
        }
        input[type="submit"]:hover {
            background-color: #45a049;
        }
    </style>
</head>
<body>

    <!-- Form for input validation -->
    <h2>Registration Form</h2>
    <form id="myForm" onsubmit="return validateForm()">
        <!-- Username field -->
        <label for="username">Username:</label>
        <input type="text" id="username" name="username">
        <div id="usernameError" class="error"></div>

        <!-- Email field -->
        <label for="email">Email:</label>
        <input type="email" id="email" name="email">
        <div id="emailError" class="error"></div>

        <!-- Password field -->
        <label for="password">Password:</label>
        <input type="password" id="password" name="password">
        <div id="passwordError" class="error"></div>

        <!-- Submit button -->
        <input type="submit" value="Submit">
    </form>

    <script>
        // JavaScript function to validate form inputs
        function validateForm() {
            // Get form inputs
            var username = document.getElementById("username").value;
            var email = document.getElementById("email").value;
            var password = document.getElementById("password").value;

            // Clear previous error messages
            document.getElementById("usernameError").innerHTML = "";
            document.getElementById("emailError").innerHTML = "";
            document.getElementById("passwordError").innerHTML = "";

            // Validation flags
            var valid = true;

            // Username validation (required)
            if (username === "") {
                document.getElementById("usernameError").innerHTML = "Username is required";
                valid = false;
            }
```

```
        // Email validation (format check)
        var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email)) {
            document.getElementById("emailError").innerHTML = "Please enter a valid email";
            valid = false;
        }

        // Password validation (required, minimum length)
        if (password === "") {
            document.getElementById("passwordError").innerHTML = "Password is required";
            valid = false;
        } else if (password.length < 6) {
            document.getElementById("passwordError").innerHTML = "Password must be at least 6 characters
long";
            valid = false;
        }

        // If any validation fails, prevent form submission
        return valid;
    }
  </script>

</body>
</html>
```

Output:

Registration Form



EXPERIMENT NO. - 5

AIM-Writing program in XML for creation of DTD, which specifies set of rules. Create a style sheet in CSS/ XSL & display the document in internet explorer.

Procedure:

XML Document Type Declaration, commonly known as DTD, is a way to describe precisely the XML language. DTDs check the validity of structure and vocabulary of an XML document against the grammatical rules of the appropriate XML language. An XML document can be defined as:

• Well-formed: If the XML document adheres to all the general XML rules such as tags must be properly nested, opening and closing tags must be balanced, and empty tags must end with '/>', then it is called as well-formed.

OR

• Valid: An XML document said to be valid when it is not only well-formed, but it also conforms to available DTD that specifies which tags it uses, what attributes those tags can contain, and which tags can occur inside other tags, among other properties.

Types:

DTD can be classified on its declaration basis in the XML document, such as:

• Internal DTD

• External DTD

When a DTD is declared within the file it is called Internal DTD and if it is declared in a separate file it is called External DTD.

We will learn more about these in the chapter DTD Syntax.

Syntax

Basic syntax of a DTD is as follows:

```
<!DOCTYPE element DTD identifier
[   declaration1
declaration2
.....
]>
```

In the above syntax

• DTD starts with <!DOCTYPE> delimiter.

• An element tells the parser to parse the document from the specified root element.

• DTD identifier is an identifier for the document type definition, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called external subset.

• The square brackets [ ] enclose an optional list of entity declarations called internal subset.

Code/Method:

```
    Employee
 2. {
3. background-color: pink;
 4. }
5. firstname,lastname,email
6. {
7.font-size:25px;
8.display:block;
9.color: blue;
10.    margin-left: 50px;
11. }
```

create the DTD file.

employee.dtd

```
<!ELEMENT employee (firstname,lastname,email)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT email (#PCDATA)>
```

employee.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="cssemployee.css"?>
<!DOCTYPE employee SYSTEM "employee.dtd">
<employee>
  <firstname>vimal</firstname>
  <lastname>jaiswal</lastname>
  <email>vimal@javatpoint.com</email>
</employee>
```

Output:

Viva Questions:

1. What is xml?
2. Define DTD.
3. Differentiate b/w internal dtd & external dtd.
4. What is xmlschema?
5. What is internal DTD**?**

EXPERIMENT NO. - 6

AIM - Create a Java Bean for Employee information (EmpID, Name, Salary, Designation and Department)

Objective:

To create a Java Bean that encapsulates employee information such as `EmpID`, `Name`, `Salary`, `Designation`, and `Department`. The bean should follow the Java Bean conventions and include getter and setter methods for accessing and modifying the fields.

Theory:

A **Java Bean** is a reusable software component written in Java. It follows specific conventions that allow for easy integration in various applications, particularly in Java EE environments. Java Beans have the following characteristics:

- **Encapsulation**: The properties of the bean are private and accessed via getter and setter methods.
- **No-argument constructor**: A default constructor is provided for easy instantiation.
- **Serializable**: The bean implements the `Serializable` interface to support saving the state of the object and later retrieving it.
- **Getter and Setter Methods**: These methods provide controlled access to the bean's properties, allowing external programs to read or update the values.

Procedure:

1. **Step 1**: Create a new Java class file named `EmployeeBean.java`.
2. **Step 2**: Define private fields to store employee details such as `EmpID`, `Name`, `Salary`, `Designation`, and `Department`.
3. **Step 3**: Create a no-argument constructor and a parameterized constructor to initialize the fields.
4. **Step 4**: Implement public getter and setter methods for each field to provide controlled access to the fields.
5. **Step 5**: Implement the `Serializable` interface to make the bean serializable.
6. **Step 6**: Write a test program to create an object of the `EmployeeBean` class and display the employee details using the getter methods.

EmployeeBean.java

```
import java.io.Serializable;

// EmployeeBean class implements Serializable interface
public class EmployeeBean implements Serializable {
    private static final long serialVersionUID = 1L;

    // Private fields for employee information
    private int empID;
    private String name;
    private double salary;
    private String designation;
    private String department;
```

```
// No-argument constructor
public EmployeeBean() {
    // Default constructor
}

// Parameterized constructor to initialize employee details
  public EmployeeBean(int empID, String name, double salary, String designation, String
 department) {
    this.empID = empID;
    this.name = name;
    this.salary = salary;
    this.designation = designation;
    this.department = department;
}

// Getter and setter for empID
public int getEmpID() {
    return empID;
}

public void setEmpID(int empID) {
    this.empID = empID;
}

// Getter and setter for name
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

// Getter and setter for salary
public double getSalary() {
    return salary;
}

public void setSalary(double salary) {
    this.salary = salary;
}

// Getter and setter for designation
public String getDesignation() {
    return designation;
}

public void setDesignation(String designation) {
    this.designation = designation;
}

// Getter and setter for department
public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
```

```
        this.department = department;
    }
}
```

EmployeeTest.java

```
public class EmployeeTest {
    public static void main(String[] args) {
        // Creating an EmployeeBean object using the parameterized constructor
        EmployeeBean employee = new EmployeeBean(101, "Aman", 50000.00, "Software Engineer",
"IT");

        // Display employee details using getter methods
        System.out.println("Employee ID: " + employee.getEmpID());
        System.out.println("Employee Name: " + employee.getName());
        System.out.println("Employee Salary: " + employee.getSalary());
        System.out.println("Employee Designation: " + employee.getDesignation());
        System.out.println("Employee Department: " + employee.getDepartment());

        // Modifying the employee salary using setter method
        employee.setSalary(55000.00);
        System.out.println("Updated Employee Salary: " + employee.getSalary());
    }
}
```

Output:

When the EmployeeTest program is run, the following output will be displayed:

Employee ID: 101
Employee Name: Aman
Employee Salary: 50000.0
Employee Designation: Software Engineer
Employee Department: IT
Updated Employee Salary: 55000.0

Viva Questions:

1. What is a Java Bean?
2. Why does a Java Bean need to implement the `Serializable` interface?
3. What is the significance of getter and setter methods in a Java Bean?
4. What is the role of the no-argument constructor in a Java Bean?
5. What is the use of `serialVersionUID` in a Serializable class?

EXPERIMENT NO. - 7

AIM - Build a command-line utility using Node.js that performs a specific task, such as converting text to uppercase, calculating the factorial of a number, or generating random passwords.

Objective:

To create a command-line utility using Node.js that performs a specific task. In this experiment, we will develop a utility that can perform one of the following operations based on user input:

1. Convert text to uppercase.
2. Calculate the factorial of a number.
3. Generate random passwords.

Theory:

Node.js is a JavaScript runtime built on Chrome's V8 engine. It is widely used for creating server-side and networking applications. With Node.js, we can also create command-line utilities that take input from users, perform specific tasks, and output the results directly to the terminal.

In this experiment, we will build a basic command-line utility using Node.js. The utility will use core modules such as `process` to handle input/output from the terminal.

Procedure:

1. **Step 1**: Install Node.js from the official website if not already installed.
2. **Step 2**: Create a new project folder for the utility.
3. **Step 3**: Initialize the project using the `npm init` command to create a `package.json` file.
4. **Step 4**: Create a JavaScript file, e.g., `cli-utility.js`, and implement the required functionality.
5. **Step 5**: Write code to handle command-line arguments and perform the following tasks:
   - Convert text to uppercase.
   - Calculate the factorial of a number.
   - Generate random passwords.
6. **Step 6**: Test the utility by running the program with different command-line arguments.

Code:

Open terminal and create a project folder:

```
mkdir cli-utility
```

```
cd cli-utility
```

Initialize the project using npm:

```
Npm init –y
```

cli-utility.js

```
#!/usr/bin/env node

// Import necessary modules
const process = require('process');

// Function to convert text to uppercase
function convertToUpper(text) {
    return text.toUpperCase();
}

// Function to calculate factorial of a number
function factorial(num) {
    if (num === 0 || num === 1) {
        return 1;
    }
    return num * factorial(num - 1);
}

// Function to generate random password
function generatePassword(length) {
    const                        charset                        =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ012345678
9";
    let password = "";
    for (let i = 0; i < length; i++) {
        const   randomIndex   =   Math.floor(Math.random()   *
charset.length);
        password += charset[randomIndex];
    }
    return password;
}

// Main function to handle command-line arguments
function main() {
    const args = process.argv.slice(2); // Get arguments passed
from command line

    if (args.length < 2) {
        console.log("Usage:");
        console.log("  uppercase <text>");
        console.log("  factorial <number>");
        console.log("  password <length>");
        process.exit(1); // Exit with failure
    }

    const command = args[0];
    const value = args[1];

    switch (command) {
        case "uppercase":
            console.log("Uppercase:", convertToUpper(value));
            break;
        case "factorial":
            const num = parseInt(value);
```

```
            if (isNaN(num) || num < 0) {
                console.log("Please provide a valid non-
negative integer.");
            } else {
                console.log(`Factorial of ${num}:`,
factorial(num));
            }
            break;
        case "password":
            const length = parseInt(value);
            if (isNaN(length) || length <= 0) {
                console.log("Please provide a valid positive
integer for password length.");
            } else {
                console.log(`Generated Password:`,
generatePassword(length));
            }
            break;
        default:
            console.log("Unknown command. Please use
'uppercase', 'factorial', or 'password'.");
    }
}

// Execute main function
main();
```

To run the utility, open the terminal in the project directory and execute the following commands based on the desired functionality:

- **Convert text to uppercase**:

```
arduino
Copy code
node cli-utility.js uppercase "hello world"
```

Output:

```
makefile
Copy code
Uppercase: HELLO WORLD
```

- **Calculate factorial**:

```
Copy code
node cli-utility.js factorial 5
```

Output:

```
mathematica
Copy code
Factorial of 5: 120
```

- **Generate random password**:

```
Copy code
node cli-utility.js password 10
```

Output:

```
yaml
Copy code
Generated Password: aZ3rT1yU0p
```

Viva Questions:

1. What is Node.js?
2. How does `process.argv` work in Node.js?
3. What is the purpose of the `#!/usr/bin/env node` line?
4. How does recursion work in the factorial calculation?
5. How does the random password generation function work?

EXPERIMENT NO. - 8

AIM - Develop a script that uses MongoDB's aggregation framework to perform operations like grouping, filtering, and sorting. For instance, aggregate user data to find the average age of users in different cities.

Objective:

To develop a Node.js script using MongoDB's Aggregation Framework to perform operations like grouping, filtering, and sorting. Specifically, the script will aggregate user data to find the average age of users in different cities.

Theory:

MongoDB Aggregation Framework is a powerful tool for performing data processing and transformation operations on documents in a collection. It allows for tasks such as filtering, grouping, sorting, and calculating averages in a flexible and efficient manner.

The basic components of MongoDB's aggregation framework include:

- **$match**: Filters documents based on conditions (similar to SQL's WHERE clause).
- **$group**: Groups documents by a specific key and allows for calculations on grouped data.
- **$sort**: Sorts the result set based on one or more fields.

MongoDB uses a **pipeline** approach for aggregation, where each stage processes the documents and passes the results to the next stage.

Procedure:

1. **Step 1**: Set up a MongoDB database and a `users` collection with relevant fields (`name`, `age`, `city`).
2. **Step 2**: Install the MongoDB Node.js driver to connect to the MongoDB database from a Node.js application.
3. **Step 3**: Write a Node.js script that connects to MongoDB and performs aggregation operations like grouping, filtering, and sorting.
4. **Step 4**: Run the script and verify the output, which should provide the average age of users in different cities.

Code/Method:

PROGRAM:

Users.json

```
[
  { "name": "John Doe", "age": 28, "city": "New York" },
  { "name": "Jane Smith", "age": 32, "city": "San Francisco" },
  { "name": "Mike Davis", "age": 21, "city": "New York" },
  { "name": "Sara Johnson", "age": 30, "city": "San Francisco" },
  { "name": "Chris Brown", "age": 25, "city": "Chicago" }
]
```

Install Dependencies

In the project folder, use npm to install the required packages:

```
npm init -y
npm install mongodb
```

aggregateUsers.js

```
const { MongoClient } = require('mongodb');

// Replace with your MongoDB connection string
const uri = 'mongodb://localhost:27017';

async function aggregateUserData() {
    const client = new MongoClient(uri);

    try {
        // Connect to the MongoDB server
        await client.connect();
        console.log("Connected successfully to MongoDB");

        // Select the database and collection
        const database = client.db('userDB');
        const users = database.collection('users');

        // Define the aggregation pipeline
        const pipeline = [
            {
                $group: {
                    _id: "$city",  // Group by city
                    averageAge: { $avg: "$age" },  // Calculate average age
                    totalUsers: { $sum: 1 }  // Count the number of users
                }
            },
            {
                $match: {
                    totalUsers: { $gte: 2 }  // Filter cities with at least 2 users
                }
            },
            {
                $sort: {
                    averageAge: 1  // Sort by average age in ascending order
                }
            }
        ];

        // Perform the aggregation
        const result = await users.aggregate(pipeline).toArray();

        // Display the results
        console.log("Average Age of Users in Each City:");
        result.forEach(doc => {
            console.log(`City: ${doc._id}, Average Age:
${doc.averageAge.toFixed(2)}, Total Users: ${doc.totalUsers}`);
        });

    } catch (err) {
        console.error(err);
    } finally {
```

```
        // Close the MongoDB connection
        await client.close();
    }
}

aggregateUserData();
```

Output:

Node aggregateUsers.js

Connected successfully to MongoDB
Average Age of Users in Each City:
City: New York, Average Age: 24.50, Total Users: 2
City: San Francisco, Average Age: 31.00, Total Users: 2

Viva Questions:

1. What is MongoDB's Aggregation Framework?
2. What are the common stages used in an aggregation pipeline?
3. How does the `$group` stage work in MongoDB?
4. What is the purpose of `$match` in an aggregation pipeline?
5. What is the role of `$sort` in the pipeline?

EXPERIMENT NO. - 9

AIM- Assume four users user1, user2, user3 and user4 having the passwords pwd1, pwd2, pwd3 and pwd4 respectively. Write a servlet for doing the following: 1. Create a Cookie and add these four user id's and passwords to this Cookie. 2. Read the user id and passwords entered in the Login form and authenticate with the values available in the cookies.

Procedure:

JSP scripting elements let you insert Java code into the servlet that will be generated from the current JSP page. There are three forms:

Expressions of the form <%= expression %> that are evaluated and inserted into the output,

Scriptlets of the form <% code %> that are inserted into the servlet's service method, and

Declarations of the form <%! code %> that are inserted into the body of the servlet class, outside of any existing methods.Each of theseis described in more detail below.JSP Expressions

A JSP expression is used to insert Java values directly into the output. It has the following form:

<%= Java Expression %>

The Java expression is evaluated, converted to a string, and inserted in the page. This evaluation is performed at run-time (when the page is requested), and thus has full access to information about the request. For example, the following showsthe date/time that the page was requested:

Current time: <%= new java.util.Date() %>

To simplify these expressions, there are a number of predefined variables that you can use. These implicit objects are discussed in more detail later, but for the purpose ofexpressions, the most important ones are:

request, the HttpServletRequest; response, the HttpServletResponse;

session, the HttpSession associated with the request (if any); and

out, the PrintWriter (a buffered version of type JspWriter) used to send output to the client.JSP Scriptlets

If you want to do something more complex than insert a simple expression, JSP scriptlets let you insert arbitrary code into the servlet method that will be built to generate the page. Scriptlets have the following form:

<% Java Code %>

Scriptlets have access to the same automatically defined variables as expressions. So, for example, if you want output to appear in the resultant page, you would use the out variable.

```
<%
      String queryData = request.getQueryString(); out.println("Attached GET data: " +
queryData);
 %>
```

Note that code inside a scriptlet gets inserted exactly as written, and any static HTML (template text) before or after a scriptlet gets converted to print statements. This means that scriptlets need not contain complete Java statements, and blocks left open can affect the static HTML outside of the scriptlets.

JSP Declarations

A JSP declaration lets you define methods or fields that get inserted into the main body of the servlet class (outside of the service method processing the request). It has the following form:

<%! Java Code %>

Since declarations do not generate any output, they are normally used in conjunction with JSP expressions or scriptlets. For example, here is a JSP fragment that prints out the number of times the current page has been requested since the server booted (or the servlet class was changed and reloaded):

<%! privateintaccessCount = 0; %>

Code/Method:

Login.html:

```
<!--Home.html-->
<html><body>
<center><h1>XYZ Company Ltd.</h1></center>
<table border="1" width="100%" height="100%">
<tr>
<td valign="top" align="center"><br/>
<form action="auth.jsp"><table>
<tr>
<td colspan="2" align="center"><b>Login Page</b></td>
</tr>
<tr>
<td colspan="2" align="center"><b> </td>
</tr>
<tr>
<td>User Name</td>
<td><input type="text" name="user"/></td>
</tr>
<tr>
<td>Password</td>
<td><input type="password" name="pwd"/></td>
</tr>
<tr>
<td> </td>

<td colspan="2" align="center"><input type="submit" value="LogIN"/></td>
<td> </td></tr><tr>

</tr>
</table>
</form>
</td>
</tr>
</table>
</body>
</html>
```

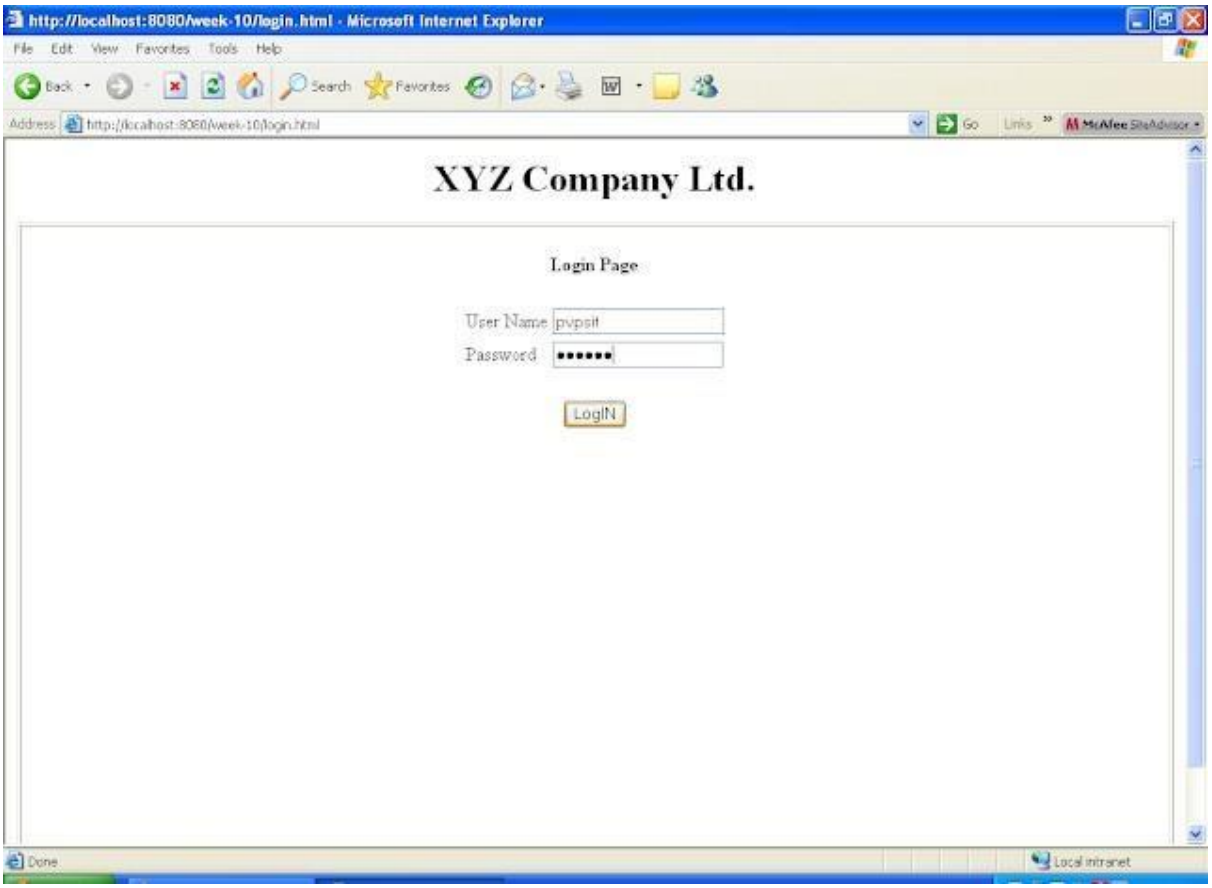
Auth.jsp:

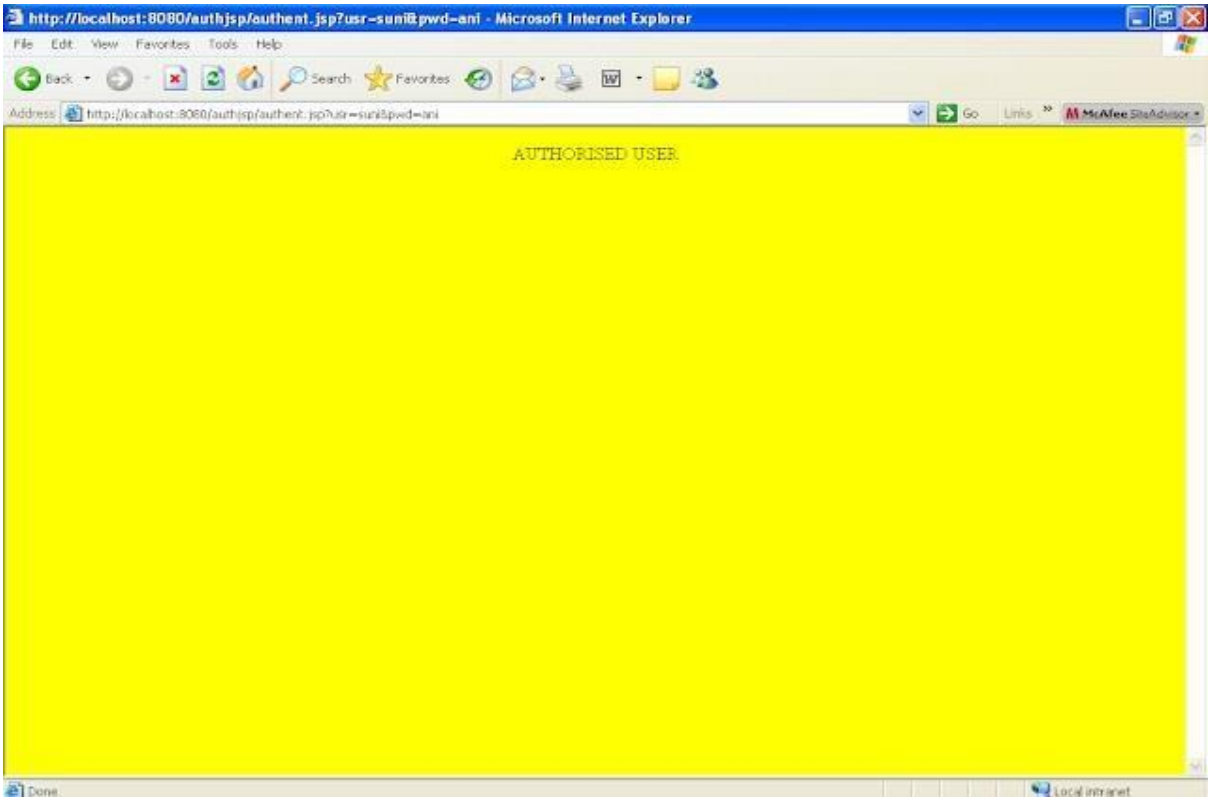
```
<%@page import="java.sql.*;"%>
```

```
<html>
<head>
<title>
This is simple data base example in JSP</title>
</title>
</head>
<body bgcolor="yellow">
<%!Stringuname,pwd;%>
<%
uname=request.getParameter("user");
pwd=request.getParameter("pwd");
try
{
Class.forName("oracle.jdbc.driver.OracleDriver");
  Connection
                con=DriverManager.getConnection("jdbc:oracle:thin:@195.100.101.158:1521
                :CCLAB","scott","tiger");
  Statement st=con.createStatement();
ResultSetrs=st.executeQuery("select name,password from personal where name='"+uname+"'
                and password='"+pwd+"'");
if(rs.next())
{
out.println("Authorized person");
}
else
{
out.println("UnAuthorized person");
}
con.close();}

catch(Exception e){out.println(""+e);}
%></body></html>
```

Output:


http://localhost:8080/week-10/login.html - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites
Address http://localhost:8080/week-10/login.html
Go Links
McAfee SiteAdvisor
XYZ Company Ltd.
Login Page
User Name pvpsit
Password
LogIN
Done
Local intranet


http://localhost:8080/authjsp/authent.jsp?usr=sunil&pwd=ani - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites
Address http://localhost:8080/authjsp/authent.jsp?usr=sunil&pwd=ani
Go Links
McAfee SiteAdvisor
AUTHORISED USER
Done
Local intranet

Viva Questions:

1. What is class?
2. Define Method and property.
3. What is Constructor?
4. Define types of Constructor.
5. Define the copy Constructor**.**

EXPERIMENT NO. - 10

AIM: Create a table which should contain at least the following fields: name, password, email-id, phone number Write Servlet/JSP to connect to that database and extract data from the tables and display them. Insert the details of the users who register with the web site, whenever a new user clicks the submit button in the registration page.

Procedure:

1. Create Maven Project
2.Click Next button to select Archetype for project
3.Click Next button and enter Project Information:
    Group Id: LearnJSPServletWithRealApps
    Artifact Id: LearnJSPServletWithRealApps
    Package: com.demo

Code/Method:

Configure pom.xml-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>LearnJSPServletWithRealApps</groupId>
    <artifactId>LearnJSPServletWithRealApps</artifactId>
    <packaging>war</packaging>
    <version>0.0.1-SNAPSHOT</version>
    <name>Learn JSP-Servlet with Real Apps</name>
    <url>http://maven.apache.org</url>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>javax.servlet.jsp.jstl</groupId>
            <artifactId>javax.servlet.jsp.jstl-api</artifactId>
            <version>1.2.1</version>
        </dependency>
        <dependency>
            <groupId>taglibs</groupId>
            <artifactId>standard</artifactId>
            <version>1.1.2</version>
        </dependency>
```

```
		<dependency>
			<groupId>javax.servlet</groupId>
			<artifactId>javax.servlet-api</artifactId>
			<version>3.1.0</version>
			<scope>provided</scope>
		</dependency>
	</dependencies>
	<build>
		<finalName>LearnJSPServletWithRealApps</finalName>
	</build>
</project>
```

Configure web.xml-

```
<!DOCTYPE web-app PUBLIC
 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
 "http://java.sun.com/dtd/web-app_2_3.dtd" >

<web-app>
	<welcome-file-list>
		<welcome-file>index.jsp</welcome-file>
	</welcome-file-list>
</web-app>
```

Redirect Page

Create new JSP file named index.jsp in src\main\webapp folder. This file will redirect to ProductServlet as below:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
	pageEncoding="ISO-8859-1"%>
<jsp:forward page="product"></jsp:forward>
```

Add Image Files

Create new folder named assets in src\main\webapp folder. In assets folder, create new folder named images and copy images need use in project to images folder.

Create Entities

Create new package named com.demo.entities. In this package, create new Java classes as below:

Product Entity

Create new Java class named Product.java as below:

```
packagecom.demo.entities;

public class Product {

	private String id;
	private String name;
	private double price;
	private String photo;
```

```
	public Product() {
	}

	public Product(String id, String name, String photo, double price) {
		this.id = id;
		this.name = name;
		this.price = price;
		this.photo = photo;
	}

	public String getId() {
		return id;
	}

	public void setId(String id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public double getPrice() {
		return price;
	}

	public void setPrice(double price) {
		this.price = price;
	}

	public String getPhoto() {
		return photo;
	}

	public void setPhoto(String photo) {
		this.photo = photo;
	}

}
```

Item Entity

Create new Java class named Item.java as below:

```
packagecom.demo.entities;

public class Item {

	private Product product;
	privateint quantity;

	public Item() {
	}

	public Item(Product product, int quantity) {
		this.product = product;
		this.quantity = quantity;
	}

	public Product getProduct() {
		return product;
	}

	public void setProduct(Product product) {
		this.product = product;
	}

	publicintgetQuantity() {
		return quantity;
	}

	public void setQuantity(int quantity) {
		this.quantity = quantity;
	}

}
```

Create Servlets

Create new package named com.demo.servlets. In this package, create new Servlets as below:

Product Servlet

In com.demo.servlets package, create new Servlet named ProductServlet as below:

```
packagecom.demo.servlets;

importjava.io.IOException;

importjavax.servlet.ServletException;
```

```
importjavax.servlet.annotation.WebServlet;
importjavax.servlet.http.HttpServlet;
importjavax.servlet.http.HttpServletRequest;
importjavax.servlet.http.HttpServletResponse;

importcom.demo.models.ProductModel;

@WebServlet("/product")
public class ProductServlet extends HttpServlet {

	private static final long serialVersionUID = 1L;

	publicProductServlet() {
		super();
	}

	protected void doGet(HttpServletRequest request, HttpServletResponse response)
			throwsServletException, IOException {
		ProductModelproductModel = new ProductModel();
		request.setAttribute("products", productModel.findAll());
		request.getRequestDispatcher("product/index.jsp").forward(request,
response);
	}

	protected void doPost(HttpServletRequest request, HttpServletResponse response)
			throwsServletException, IOException {
	}

}
```

Cart Servlet

In com.demo.servlets package, create new Servlet named CartServlet as below:

```
packagecom.demo.servlets;

importjava.io.IOException;
importjava.util.ArrayList;
importjava.util.List;

importjavax.servlet.ServletException;
importjavax.servlet.annotation.WebServlet;
importjavax.servlet.http.HttpServlet;
importjavax.servlet.http.HttpServletRequest;
importjavax.servlet.http.HttpServletResponse;
importjavax.servlet.http.HttpSession;
```

```
importcom.demo.entities.Item;
importcom.demo.models.ProductModel;

@WebServlet("/cart")
public class CartServlet extends HttpServlet {

	private static final long serialVersionUID = 1L;

	publicCartServlet() {
		super();
	}

	protected void doGet(HttpServletRequest request, HttpServletResponse response)
			throwsServletException, IOException {
		String action = request.getParameter("action");
		if (action == null) {
			doGet_DisplayCart(request, response);
		} else {
			if (action.equalsIgnoreCase("buy")) {
				doGet_Buy(request, response);
			} else if (action.equalsIgnoreCase("remove")) {
				doGet_Remove(request, response);
			}
		}
	}

	protected void doGet_DisplayCart(HttpServletRequest request,
HttpServletResponse response)
			throwsServletException, IOException {
		request.getRequestDispatcher("cart/index.jsp").forward(request, response);
	}

	protected void doGet_Remove(HttpServletRequest request, HttpServletResponse
response)
			throwsServletException, IOException {
		HttpSession session = request.getSession();
		List<Item> cart = (List<Item>) session.getAttribute("cart");
		int index = isExisting(request.getParameter("id"), cart);
		cart.remove(index);
		session.setAttribute("cart", cart);
		response.sendRedirect("cart");
	}

	protected void doGet_Buy(HttpServletRequest request, HttpServletResponse
response)
```

```
                throwsServletException, IOException {
            ProductModelproductModel = new ProductModel();
            HttpSession session = request.getSession();
            if (session.getAttribute("cart") == null) {
                List<Item> cart = new ArrayList<Item>();
                cart.add(new  Item(productModel.find(request.getParameter("id")),
1));
                session.setAttribute("cart", cart);
            } else {
                List<Item> cart = (List<Item>) session.getAttribute("cart");
                int index = isExisting(request.getParameter("id"), cart);
                if (index == -1) {
                    cart.add(new
Item(productModel.find(request.getParameter("id")), 1));
                } else {
                    int quantity = cart.get(index).getQuantity() + 1;
                    cart.get(index).setQuantity(quantity);
                }
                session.setAttribute("cart", cart);
            }
            response.sendRedirect("cart");
        }

        privateintisExisting(String id, List<Item> cart) {
            for (int i = 0; i <cart.size(); i++) {
                if (cart.get(i).getProduct().getId().equalsIgnoreCase(id)) {
                    return i;
                }
            }
            return -1;
        }

        protected void doPost(HttpServletRequest request, HttpServletResponse response)
                throwsServletException, IOException {
        }

}
```

Create Product Pages

Create new folder named product in src\main\webapp folder. In this folder, create new JSP file named index.jsp as below:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1" isELIgnored="false"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
```

```
<head>
<title>Product Page</title>
</head>
<body>

	<table cellpadding="2" cellspacing="2" border="1">
		<tr>
			<th>Id</th>
			<th>Name</th>
			<th>Photo</th>
			<th>Price</th>
			<th>Buy</th>
		</tr>
		<c:forEachvar="product" items="${products }">
			<tr>
				<td>${product.id }</td>
				<td>${product.name }</td>
				<td>
					<imgsrc="${pageContext.request.contextPath
}/assets/images/${product.photo }" width="120">
				</td>
				<td>${product.price }</td>
				<td align="center">
					<a	href="${pageContext.request.contextPath
}/cart?&action=buy&id=${product.id }">Buy</a>
				</td>
			</tr>
		</c:forEach>
	</table>

</body>
</html>
```

Create Cart Pages

Create new folder named cart in src\main\webapp folder. In this folder, create new JSP file named index.jsp as below:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
	pageEncoding="ISO-8859-1" isELIgnored="false"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
<head>
<title>Cart Page</title>
</head>
<body>
```

```
<table cellpadding="2" cellspacing="2" border="1">
	<tr>
		<th>Option</th>
		<th>Id</th>
		<th>Name</th>
		<th>Photo</th>
		<th>Price</th>
		<th>Quantity</th>
		<th>Sub Total</th>
	</tr>
	<c:setvar="total" value="0"></c:set>
	<c:forEachvar="item" items="${sessionScope.cart }">
		<c:setvar="total" value="${total + item.product.price * item.quantity }"></c:set>
		<tr>
			<td align="center">
				<a href="${pageContext.request.contextPath }/cart?action=remove&id=${item.product.id }"
				onclick="return confirm('Are you sure?')">Remove</a>
			</td>
			<td>${item.product.id }</td>
			<td>${item.product.name }</td>
			<td>
				<imgsrc="${pageContext.request.contextPath }/assets/images/${item.product.photo }" width="120">
			</td>
			<td>${item.product.price }</td>
			<td>${item.quantity }</td>
			<td>${item.product.price * item.quantity }</td>
		</tr>
	</c:forEach>
	<tr>
		<td colspan="6" align="right">Total</td>
		<td>${total }</td>
	</tr>
</table>
<br>
<a href="${pageContext.request.contextPath }/product">Continue Shopping</a>

</body>
</html>
```

Output:

Id	Name	Photo	Price	Buy
p01	name 1		20.0	Buy
p02	name 2		21.0	Buy
p03	name 3		22.0	Buy

Viva Questions:
1. What is Servlet?
2. What is JSP?
3. What are uses of XML file?
4. Define JSP Tag.
5. Define Implicit object.

EXPERIMENT NO. - 11

AIM- Write a JSP which insert the details of the 3 or 4 users who register with the web site by using registration form. Authenticate the user when he submits the login form using the user name and password from the database.

Procedure:

Code/Method:

```
packagecom.cse;
public class JumpString {

        public String conCat(String st1, String st2)
        {
        if (st1.length() != 0 && st2.length() != 0
        &&st1.charAt(st1.length() - 1) == st2.charAt(0))
        return st1 + st2.substring(1);
        return st1 + st2;
        }

        public static void main (String[] args)
           {
                JumpString m= new JumpString();
             String str1 =  "food";
             String str2 = "door";
        System.out.println("The given strings are: "+str1+" and "+str2);
        System.out.println("The string after concatination are: "+m.conCat(str1,str2));
           }
}
```

Output:

The given strings are: food and door
The string after concatination are:

VIVA Questions:

1. What is java string?
2. Define Reverse String.
3. Define concatenate string.
4. What is java package?
5. What are the difference between class & packages?

EXPERIMENT NO. - 12

AIM- Design and implement a simple shopping cart example with session tracking API.

Procedure:

Objective:

To design and implement a simple shopping cart application using Node.js and Express.js that allows users to add items to their cart, view the cart contents, and manage session tracking to maintain the cart's state.

Theory:

A shopping cart is a crucial component of any e-commerce application, allowing users to select and review products before making a purchase. In this experiment, we will build a basic shopping cart application that utilizes sessions to keep track of the user's cart contents.

Key Concepts:

- **Node.js**: A JavaScript runtime used for building server-side applications.
- **Express.js**: A web application framework for Node.js that simplifies routing and middleware management.
- **Sessions**: Mechanisms to store user data on the server for the duration of their visit.

We will use the `express-session` middleware to handle session tracking, allowing us to persist user data across different routes.

Procedure:

1. **Step 1**: Set up a new Node.js project.
2. **Step 2**: Install the required packages.
3. **Step 3**: Create an Express.js server and implement routes for adding items to the cart, viewing the cart, and removing items from the cart.
4. **Step 4**: Test the API endpoints using Postman.

Code/Method:

Step 1: Set Up a New Node.js Project

1. Create a new directory for the shopping cart application:

   mkdir shopping-cart

   cd shopping-cart

2. Initialize a new Node.js project:

   npm init –y

Step 2: Install Required Packages

Install the necessary packages using npm:

npm install express express-session body-parser

Step 3: Create the Express.js Server

Create a file named `app.js` and write the following code:

```
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');

const app = express();
const PORT = 3000;

// Middleware
app.use(bodyParser.json());
app.use(session({
   secret: 'your_secret_key',  // Replace with a secure key in production
   resave: false,
   saveUninitialized: true,
   cookie: { secure: false }  // Set to true if using HTTPS
}));

// Sample product data (in a real application, this could be retrieved from a database)
const products = [
   { id: 1, name: 'Laptop', price: 50000 },
   { id: 2, name: 'Mobile Phone', price: 15000 },
   { id: 3, name: 'Headphones', price: 2000 }
];

// Initialize shopping cart if it doesn't exist
app.use((req, res, next) => {
   if (!req.session.cart) {
      req.session.cart = [];
   }
   next();
});

// Route to view available products
app.get('/products', (req, res) => {
   res.json(products);
```

```
});

// Route to add an item to the cart
app.post('/cart/add', (req, res) => {
    const productId = req.body.productId;
    const product = products.find(p => p.id === productId);

    if (product) {
        req.session.cart.push(product);
        res.json({ message: 'Product added to cart', cart: req.session.cart });
    } else {
        res.status(404).json({ message: 'Product not found' });
    }
});

// Route to view cart contents
app.get('/cart', (req, res) => {
    res.json(req.session.cart);
});

// Route to remove an item from the cart
app.post('/cart/remove', (req, res) => {
    const productId = req.body.productId;
    req.session.cart = req.session.cart.filter(p => p.id !== productId);
    res.json({ message: 'Product removed from cart', cart: req.session.cart });
});

// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Step 4: Testing the API

1. **Start the Server**: Run the following command in the terminal:

   ```
   node app.js
   ```

2. **Use Postman to Test the API**:

- **View Available Products**:
  - Method: `GET`
  - URL: `http://localhost:3000/products`

Expected Response:

```
[
   { "id": 1, "name": "Laptop", "price": 50000 },
   { "id": 2, "name": "Mobile Phone", "price": 15000 },
   { "id": 3, "name": "Headphones", "price": 2000 }
]
```

Viva Questions:

1. What is the purpose of using sessions in a web application?
2. How does the `express-session` middleware work?
3. What are some advantages of using JSON for request and response bodies?
4. What would you do to make the shopping cart application production-ready?
5. How can you extend this application to support user authentication?